

Applied NLP

DATA 641

Lecture 7 — CNNs for Text

*Practical Natural Language Processing: A Comprehensive Guide to Building Real-World NLP Systems 1st Edition, Sowmya Vajjala, Bodhisattwa Majumder, Anuj Gupta, Harshit Surana, O' Reilly and Natural Language Processing in Action, Understanding, analyzing, and generating text with Python, Hobson Lane, Cole Howard, Hannes Hapke, First edition, MANNING

*Figures are from the lecture notes: https://lena-voita.github.io/nlp_course

One-dimensional filters

It turns out you can use convolutional neural networks for natural language processing by using word vectors (also known as word embeddings).

- Because relative vertical relations between words would be arbitrary, depending on the page width, no relevant information is in the patterns that may emerge there.
- Relevant information is in the relative "horizontal" positions though.

1 × 3 Filter

The cat and dog went to the bodega together.

1 × 3 Filter

The cat and dog went to the bodega together.

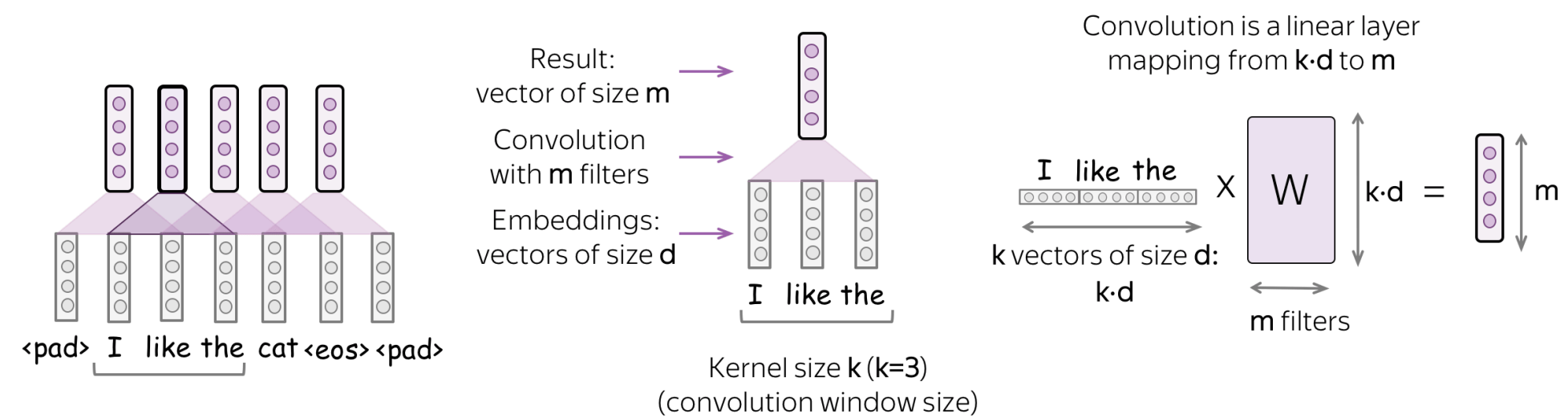
1 × 3 Filter

The cat and dog went to the bodega together.

***The same concepts hold true for languages that are read top to bottom before reading right or left, such as Japanese. But in those cases, you focus on "vertical"**

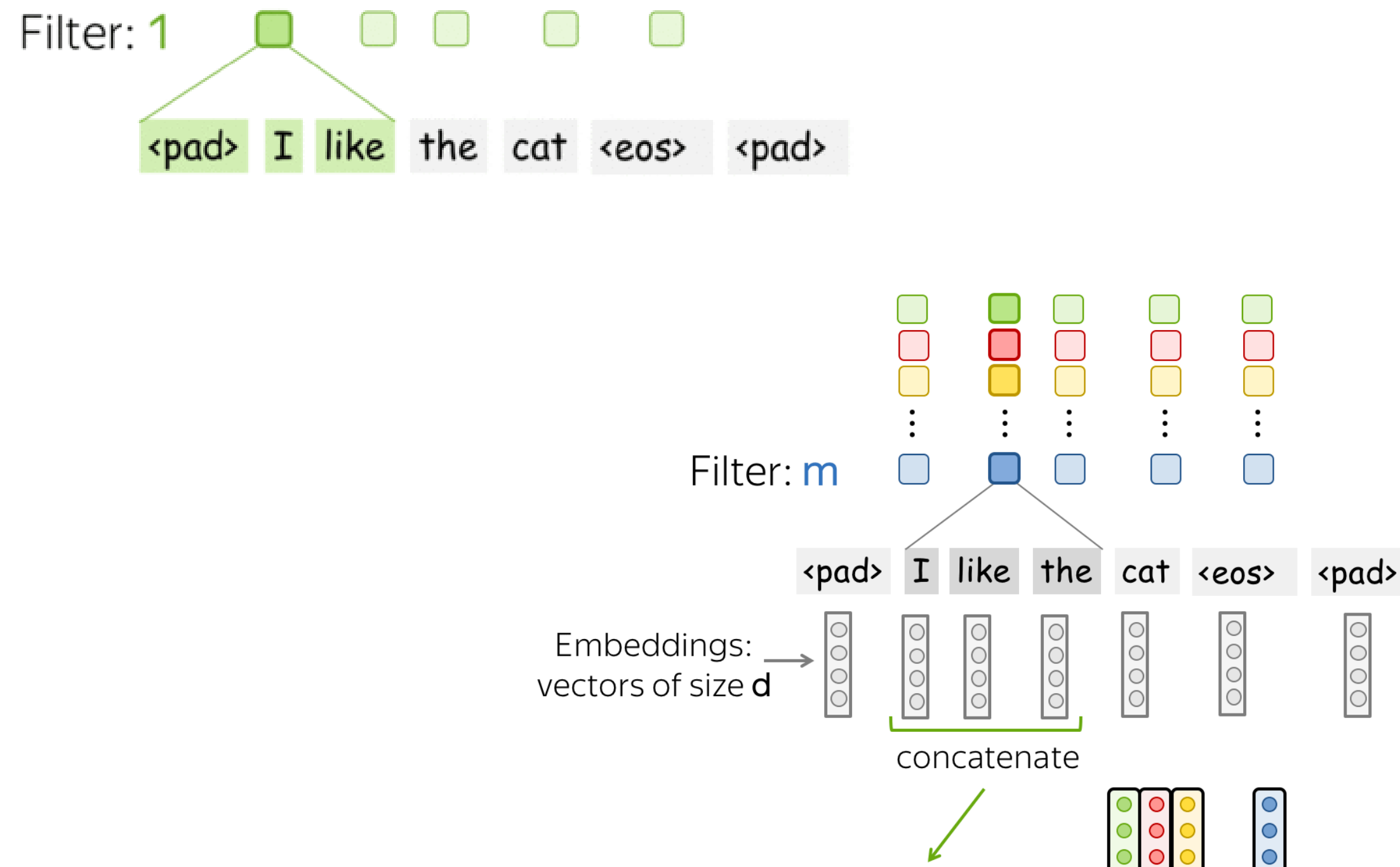
One-dimensional filter for word embeddings

If you imagine the text as an image, the "second" dimension is the full length of the word vector, typically 100-D--500-D, just like a real image.



Note 1: The term one-dimensional filter can be a little misleading as you get to word embeddings. The vector representation of the word itself extends "downward", but the filter covers the whole length of that dimension in one go. The dimension we are referring to when we say one dimensional convolution is the "width" of the phrase—the dimension we are traveling across.

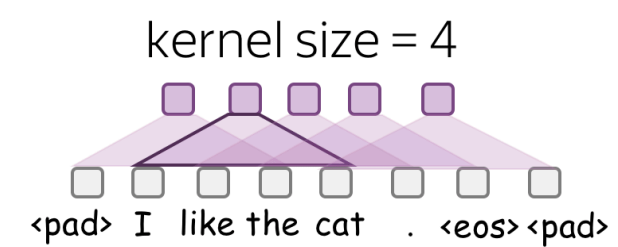
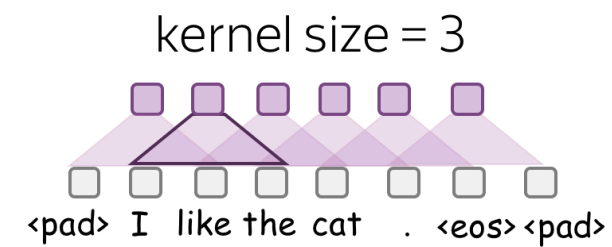
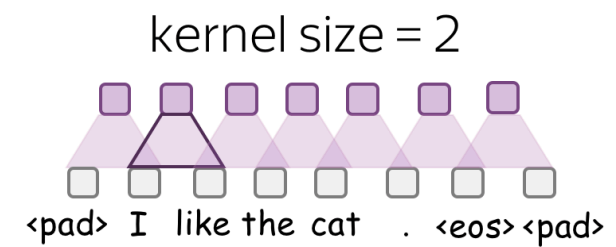
Note 2: One filter extracts a single feature. Usually, we want many features: for this, we have to take several filters. Each filter reads an input text and extracts a different feature. The number of filters is the number of output features you want to get.



Parameters: Kernel size, Stride, Padding, Bias

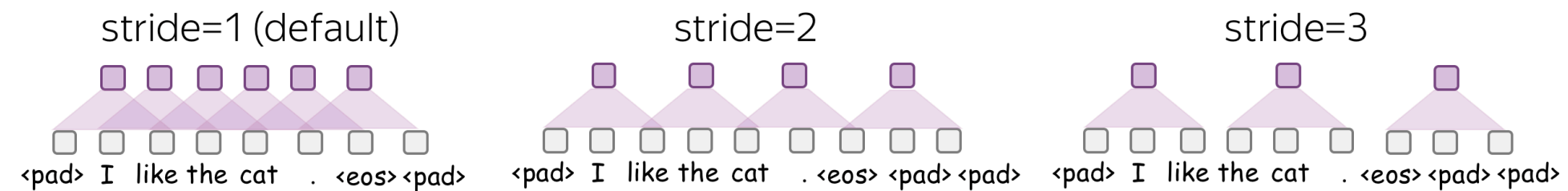
Kernel size: How far to look

Kernel size is the number of input elements (tokens) a convolution looks at each step. For text, typical values are 2-5.



Stride: How much move a filter at each step

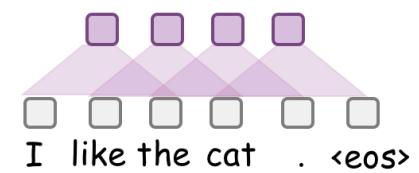
Stride tells how much to move filter at each step. For example, stride equal to 1 means that we move the filter by 1 input element (pixel for images, token for texts) at each step.



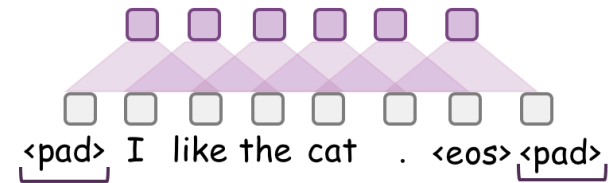
Padding: Add zero vectors to both sides

Padding adds zero vectors to both sides of an input. If you are using $\text{stride} > 1$, you may need padding.

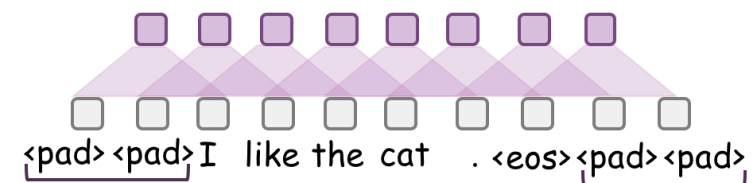
padding=0 (default)



padding=1

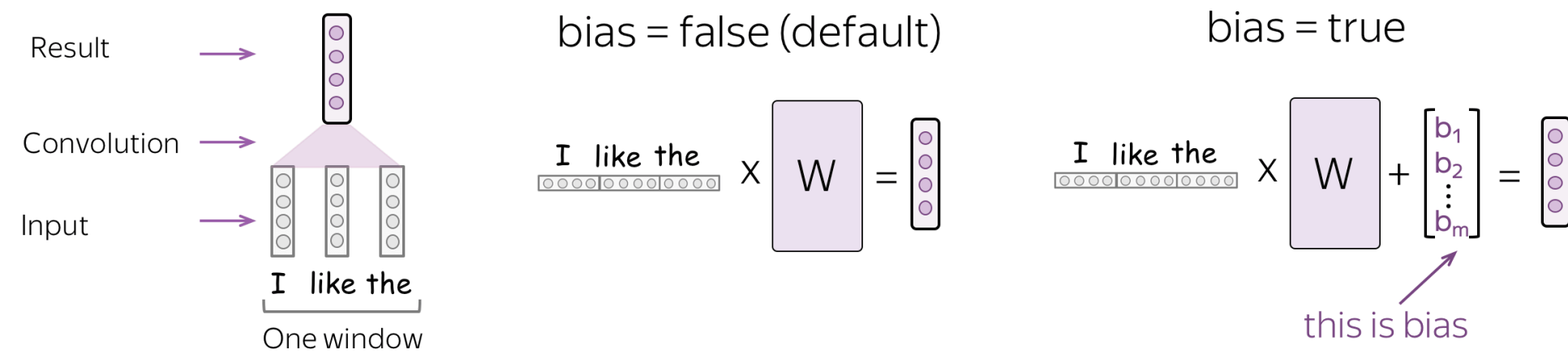


padding=2



Bias: The bias term in the linear operation in convolution.

By default, there is no bias - only multiplication by a matrix.



Building Blocks: Pooling

After a convolution extracted several features from each window, a pooling layer summarizes the features in some region. Pooling layers are used to reduce the input dimension, and, therefore, to reduce the number of parameters used by the network.

Max and Mean Pooling

